# Retail Analytics Bot

*Project Architecture & Implementation Report*

## 1. Project Overview & Database Background

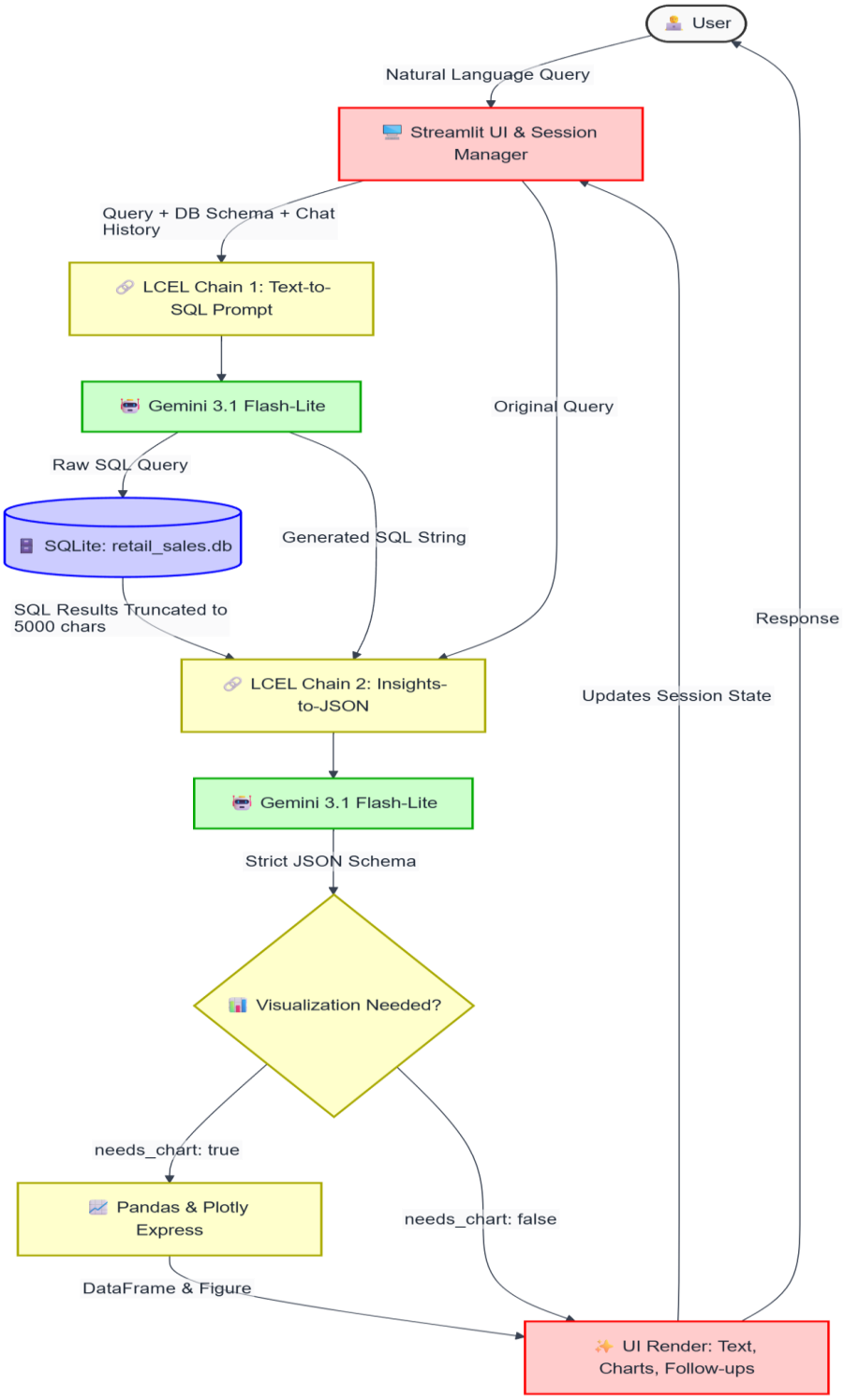
This project details the architecture and implementation of a Retail Analytics Bot built with Streamlit, LangChain, and Google's Gemini LLM. As foundational context, the underlying database (**retail_sales.db**) was established within a Jupyter Notebook environment. Initial Exploratory Data Analysis (EDA) was performed on a source Excel file (.xlsx), which was subsequently cleaned and exported into an SQLite3 database. This local DB serves as the operational data warehouse for the chatbot's real-time querying capabilities.

## 2. System Architecture Diagram & Flow

The system relies on a two-pass Large Language Model (LLM) processing architecture managed by LangChain Expression Language (LCEL) chains, fully integrated into a Streamlit frontend.

Architectural Flow Sequence:

1. User Interface: The user submits a natural language data query via the Streamlit frontend.
2. Chain 1 (Text-to-SQL): The user query, database schema, and historical chat context are injected into the Gemini model to generate a raw SQLite query.
3. Database Execution: The generated SQL query is executed directly against the local SQLite database.
4. Chain 2 (Insights-to-JSON): The executed data results, original query, and generated SQL are sent back to the LLM to formulate a strict JSON response containing a text summary, visualization parameters, and follow-up questions.
5. UI Rendering & Feedback: Streamlit parses the JSON, displays the analyst summary, generates dynamic Plotly charts (if requested), and updates the multi-session state for continued interaction.



## 3. Data Processing Methods

Several data processing layers and safeguards are implemented to ensure pipeline stability:

- Database Execution & Truncation: Queries are scrubbed via regex to remove markdown artifacts. Results exceeding 5,000 characters are aggressively truncated with a system warning to prevent LLM context window overflow.
- Text Formatting: The 'clean_mashed_text' utility utilizes regular expressions to normalize alphanumeric spacing and correct punctuation boundaries from the LLM's raw output.
- Data Visualization Processing: Raw SQL results are ingested into Pandas DataFrames. Fallback logic is implemented to automatically assign the first and last columns to the X and Y axes in the event the LLM hallucinates column names during the JSON generation phase.

## 4. AI Model Usage

The application leverages the gemini-3.1-flash-lite model via ChatGoogleGenerativeAI, utilizing a temperature setting of 0 for highly deterministic, repeatable outputs.

- SQL Generation Prompt: Instructs the model to output ONLY raw SQL, strictly enforcing GROUP BY clauses for analytical aggregations and a LIMIT 50 constraint for raw data lookups.
- JSON Structuring Prompt: Forces the LLM to act as a structured data analyst. It maps the output to a strict schema containing keys for text_summary, needs_chart, chart_type, x_column, y_column, and an array of precisely two follow_up_questions.
- Caching Strategy: The application uses @st.cache_data with a 3600-second TTL to cache LLM responses for identical queries, significantly reducing API latency and token consumption.

## 5. UI Functionality

The Streamlit frontend is configured with a 'wide' layout and features advanced session management suited for complex data investigations:

- Multi-Session State: Employs a nested dictionary structure within st.session_state to track multiple concurrent chat histories. Users can switch between active investigation threads dynamically via the sidebar.
- Dynamic Plotly Charts: Renders Bar, Line, Scatter, and Box plots natively within the chat flow using plotly.express, keyed uniquely by session ID and message index to prevent rendering collisions.
- Contextual Interaction: Generates dynamic action buttons for LLM-suggested follow-up queries, allowing for a seamless, click-driven conversational analytics experience.
- Notebook Export Utility: Translates the active chat session into a downloadable Jupyter Notebook (.ipynb) payload, embedding both the conversational markdown history and

the raw Python/Plotly code required to regenerate the specific dataframe visualizations locally.